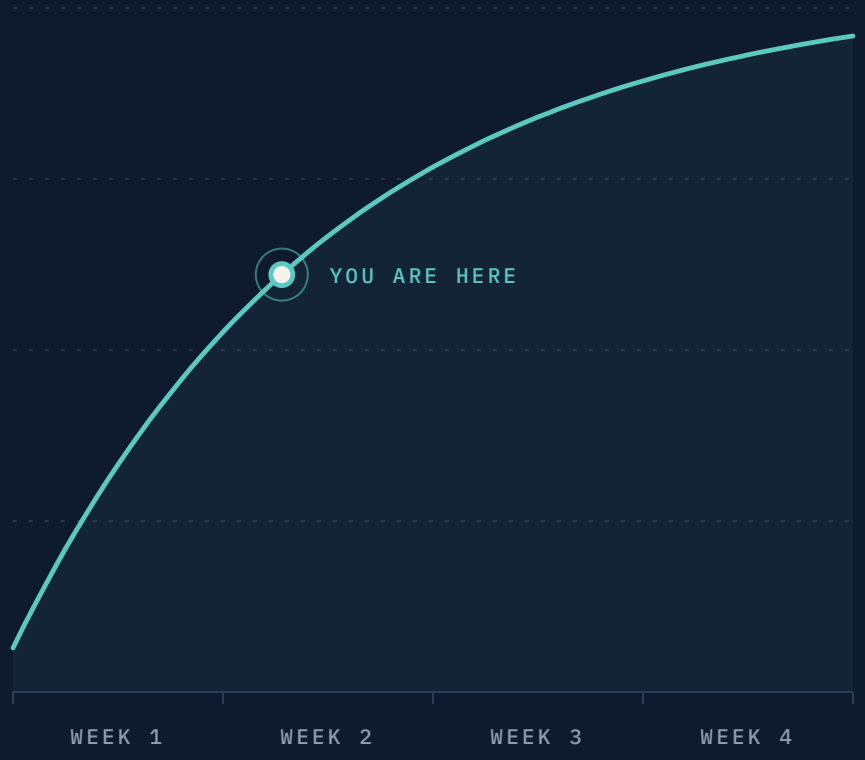
WEEK 2 · DAY 1, PART 2 · OVERVIEW

NumPy, Part 2



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Ninth Note

This morning's answers covered whole columns. These are about part of one: which bonds are below par, which are high yield, and what if the cheap ones rally a point. Then check our 98.50 on QVNE against its cash flows, value the book for Monday to Thursday, and save it all for tomorrow.

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

Your Route Through Day 1, Part 2

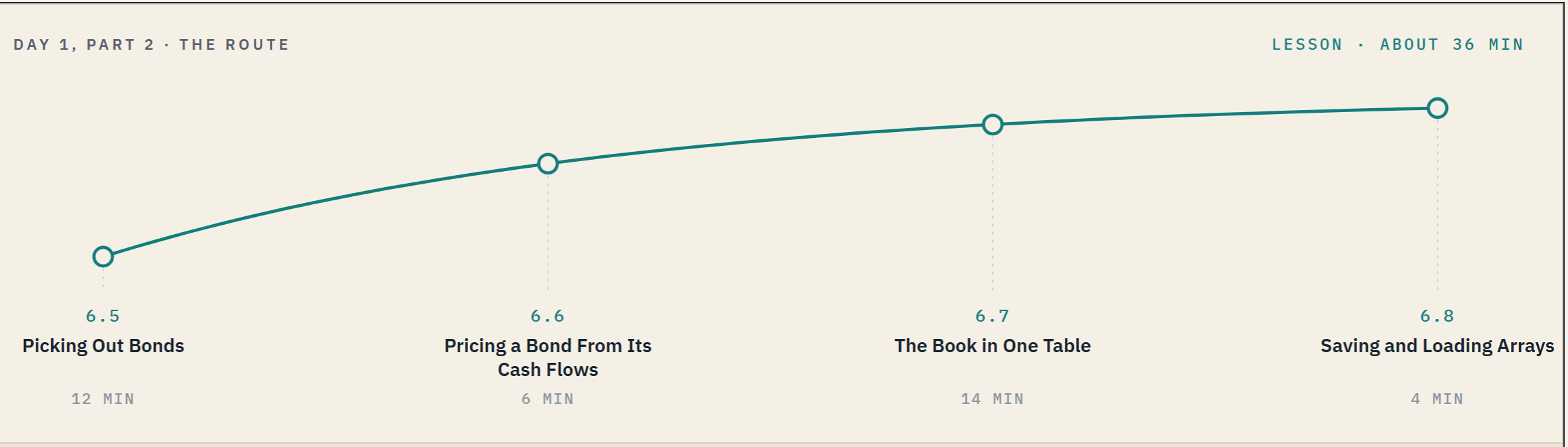


FIGURE 1 Four sections, in order, with about how long each takes.

Time. About 36 minutes for the lesson, then about 15 for the five exercises.

6.5 Picking Out Bonds

One test per bond, then the mask picks from any array

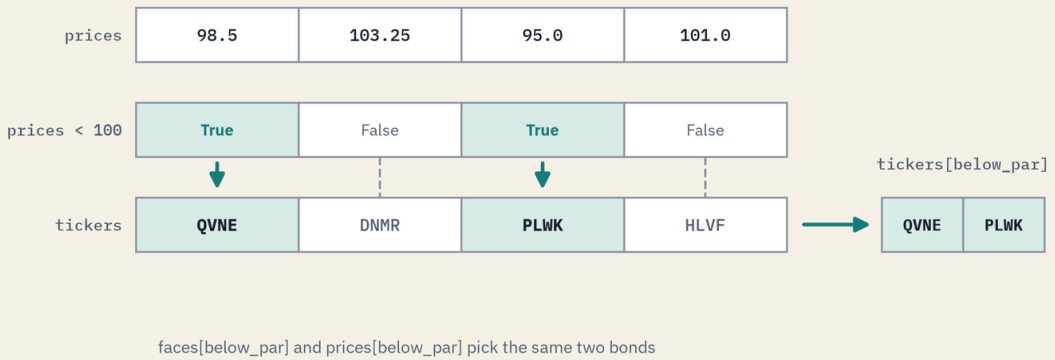


FIGURE 3 prices tested against 100 gives True, False, True, False; the same mask picks QVNE and PLWK out of tickers.

A COMPARISON GIVES A MASK

```
below_par = prices < 100
print(below_par)
```

```
[ True False  True False]
```

THE MASK PICKS THE BONDS

```
print(tickers[below_par])
print(prices[below_par])
print('Bonds below par:', below_par.sum())
```

```
['QVNE' 'PLWK']
[98.5 95. ]
Bonds below par: 2
```

One comparison, four answers, one bond order.

Combining Masks

Two tests per bond: & needs both, | needs either

	scores >= 11	prices > 102	&	prices < 96	|
QVNE	False	False	False	False	False
DNMR	True	True	True	False	True
PLWK	False	False	False	True	True
HLVF	True	False	False	False	True

HLVF fails & at 101; PLWK passes | on its price alone

FIGURE 4 Two tests per bond: and needs both True, so only DNMR passes; or needs either, so DNMR, PLWK and HLVF pass.

BOTH TESTS, OR EITHER

```
print(tickers[(scores >= 11) & (prices > 102)])
print(tickers[(scores >= 11) | (prices < 96)])
```

```
['DNMR']
['DNMR' 'PLWK' 'HLVF']
```

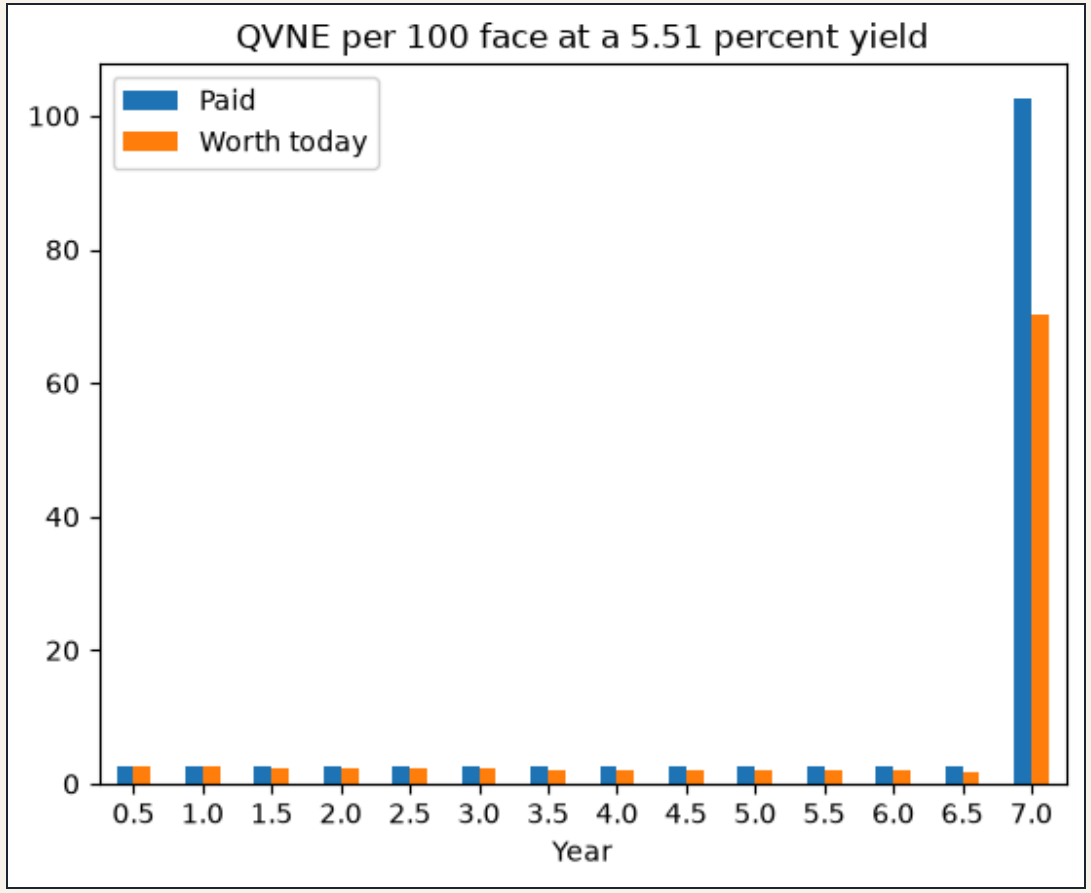
KEEP THE REAL NUMBERS

```
real = np.isfinite(yields_with_zero)
print(real)
print(yields_with_zero[real])
```

```
[ True False  True  True]
[5.32994924 5.         7.42574257]
```

Week 1 wrote `and` **and** `or` **as words. Try one.**

6.6 Pricing a Bond From Its Cash Flows



A DISCOUNT FACTOR FOR EACH PERIOD

```
yield_per_period = yield_pct / 100 / 2
discount_factors = 1 / (1 + yield_per_period) ** periods
print(np.round(discount_factors, 4))
```

WORTH TODAY, THEN ADDED UP

```
present_values = cash_flows * discount_factors
qvne_price = present_values.sum()
print('QVNE at 5.51 percent:', round(qvne_price, 4))
```

```
QVNE at 5.51 percent: 98.5067
```

98.5067: within a cent of the desk's 98.50, with no loop.

6.7 The Book in One Table

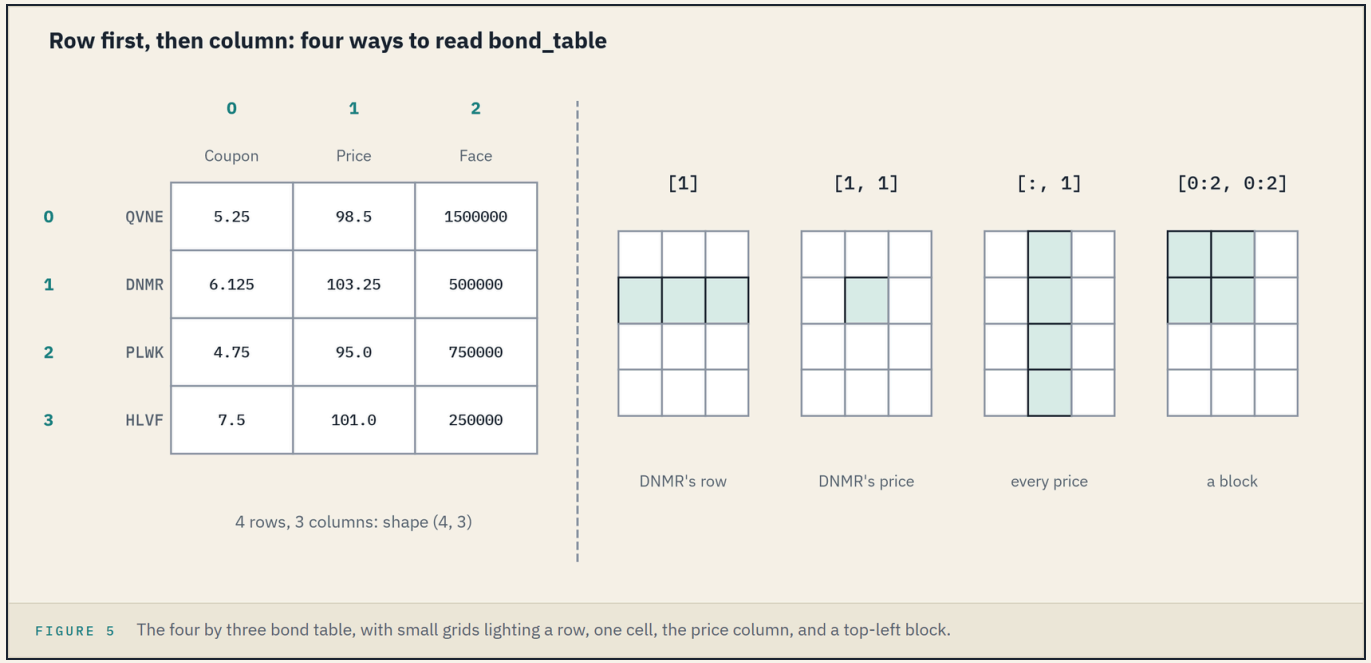


FIGURE 5 The four by three bond table, with small grids lighting a row, one cell, the price column, and a top-left block.

A ROW, A PRICE

```
print(bond_table[1])
print(bond_table[1, 1])
```

```
[     6.125    103.25  500000.   ]
103.25
```

A COLUMN, A BLOCK

```
print(bond_table[:, 1])
print(bond_table[0:2, 0:2])
```

```
[ 98.5  103.25  95.   101.  ]
[[  5.25   98.5  ]
 [  6.125 103.25 ]]
```

Row first, then column; a colon alone takes them all.

Reshape and Turn the Grid

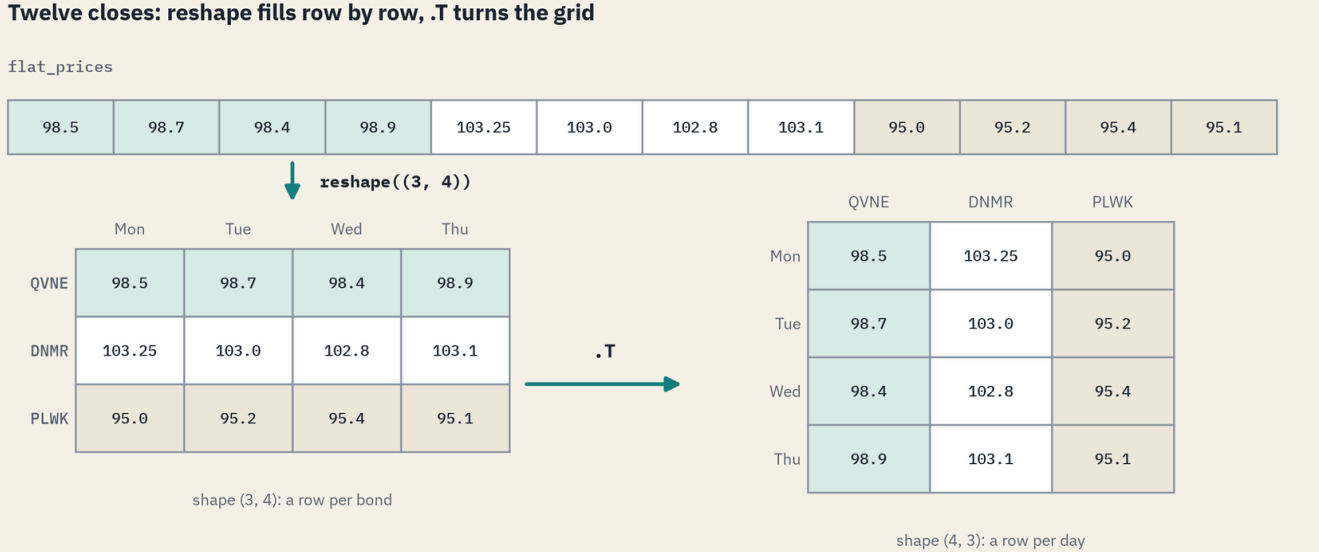


FIGURE 7 Twelve closes fill a three by four grid, a row per bond; the transpose turns it into a row per day.

TWELVE CLOSES INTO A GRID

```
price_grid = flat_prices.reshape((3, 4))
print(price_grid)
```

```
[[ 98.5   98.7   98.4   98.9 ]
 [103.25 103.   102.8  103.1 ]
 [ 95.    95.2   95.4   95.1 ]]
```

TURN THE GRID

```
print(price_grid.T)
print(price_grid.T.shape)
```

```
[[ 98.5  103.25  95.  ]
 [ 98.7  103.    95.2 ]
 [ 98.4  102.8   95.4 ]
 [ 98.9  103.1   95.1 ]]
(4, 3)
```

6.8 Saving and Loading Arrays

ONE ARRAY, ONE FILE

```
np.save('desk_prices.npy', prices)
loaded_prices = np.load('desk_prices.npy')
print(loaded_prices)
```

```
[ 98.5  103.25  95.   101.  ]
```

SEVERAL ARRAYS, ONE FILE

```
np.savez('desk_arrays.npz', prices=prices, faces=faces)
with np.load('desk_arrays.npz') as saved:
    print(saved.files)
    print(saved['faces'])
```

```
['prices', 'faces']
[1500000  500000  750000  250000]
```

THE THREE FILE TYPES

FILE	HOLDS	EVERY DIGIT KEPT	EXCEL OPENS IT
`.npy`	one array	yes	no
`.npz`	several arrays, by name	yes	no
`.csv`	one table of numbers, as text	only the decimals written	yes

Saved and read back, numbers only. Names are pandas' job.

You Can Now

- Pick bonds by position or by a mask, such as below par.
- Combine masks with `&` and `|`, and label bonds with `np.where()`.
- Change part of an array through a mask or a slice, on a `.copy()`.
- Price a bond from its cash flows, and test it at a yield equal to the coupon.
- Hold the book in a 2-D array and work along an axis, with `reshape`, `.T`, and `@`.
- Save arrays to files and check that they come back intact.

NEXT
WEEK 2 · DAY 2, PART 1

pandas: Series and DataFrames, Part 1

Holding a whole portfolio in one table, and pulling out the rows and columns a question needs.